

Evaluation of Topology-Aware All-Reduce Algorithm for Dragonfly Vs Dragonfly+ Networks

Abstract

In the era of machine learning, where massive datasets and compute-intensive models dominate, the design of data center topology plays a critical role in enabling efficient parallel processing. As workloads become increasingly distributed across GPUs, TPUs, and other accelerators, the underlying network architecture must support high-throughput, low-latency communication between nodes. Poorly chosen topologies can lead to bottlenecks, increased training times, and reduced scalability. Conversely, optimized topologies—such as fat-tree, dragonfly, or torus—can significantly enhance performance by minimizing congestion and maximizing bandwidth utilization. Moreover, fault tolerance and load balancing become essential in sustaining long-running training jobs. Therefore, selecting the right physical and logical topology is not merely a hardware decision but a strategic one that directly impacts the effectiveness and cost-efficiency of machine learning infrastructure. In addition, overall performance (throughput and latency) is effected not only by physical and logical topologies but also by the learning algorithm.

Mininet is an open-source network emulator that creates a realistic virtual network on a single Linux machine. It's widely used for Software-Defined Networking (SDN) research and prototyping. Runs real Linux network stack and processes. Mininet is Lightweight as it uses Linux namespaces and virtual Ethernet pairs to create virtual hosts, switches, and links. Custom topologies like Tree, Mesh, Ring or custom topologies can be built with Python scripts.

Project objective

1. In this project you will evaluate to performance of several physical topologies of a hypothetical data center, reconstructing a research paper recently published.

Project overview

1. Read the research papers (refs [2], [3]) and prepare to present it.
Pay a special attention to
 - 1.1. Dragonfly and Dragonfly+ topologies.
 - 1.2. The g-PAARD algorithm
2. Choose your tools (Mininet is usually a preferred option, but it is not mandatory you can use containers for routers implementation. **Document your decision process.**
3. Implement Dragonfly and Dragonfly+ topologies
Design goal: Both networks have (more or less) similar resources
 - 3.1. number of switches

3.2. aggregated bandwidth

3.3. number of links

In cases where the networks are different, document the differences and your design decisions.

4. Implement the g-PAARD all-reduced algorithm on the Dragonfly topology and adapt it for Dragonfly+ topology.
5. Compare Dragonfly to Dragonfly+ in terms of:
 - 5.1. FCT - Flow Completion Time
 - 5.2. Per-link bandwidth utilization
 - 5.3. Per-packet latency statistics
 - 5.4. Per-flow latency statistics
6. Setup Environment
 - Install Mininet (if this is your choice) on Ubuntu (your own station or on VM(s) in the CS faculty supplied by the lab)
 - Install measurement tools (iperf, tc).
7. Build Network Topologies
 - Create Python scripts for the chosen topologies
 - You can use MiniEdit for custom designs.
8. Design the scenarios for test
9. Run Experiments
 - Execute the tests
 - Measure FCT/ latency/ throughput/ link utilization ...
10. Analyze Results
 - Visualize results with graphs.
 - Compare your results to the published paper results
 - Discuss the similarities and differences

Tools You Might Use

- **Mininet**: To emulate the network.
- **Python**
- **MiniEdit**: For topology edit

Project Schedule and Milestones (Tentative)

1. Week 1 – Kickoff meeting
2. Week 3 – CDR
The students present their project roadmap:
 - a. Theoretical Review.
 - b. Requirements (what will be presented at the final presentation)
 - c. High level Design of what you plan to do
 - d. Criteria for:
 - i. Success
 - ii. Extraordinary success(This is related to the project goals)
 - e. Schedule – verify that you have enough time to accomplish what you plan to do.
3. Week 11 - Dry Run before Final Presentation
 - a. PowerPoint Presentation (In person)
 - b. Final Report – Draft
 - c. Project Poster – Draft
4. Week 12 - Final Presentation
 - a. PowerPoint Presentation (In person)
 - b. Final Report – Draft
 - c. Project Poster – Draft
5. Deliveries:
 - a. Final Report (docx)
 - b. Final Presentation (pptx)
 - c. Project Poster (pptx)

Notes

1. Progress and sync meetings will be handled weekly (or as the instructor decides) in person or zoom.
2. The above list is an estimate. Goals and tasks may be modified during the first few weeks of the semester (before CDR).
3. General requirements for all LCCN Projects are specified at the lab webcourse.

Prerequisites:

1. Introduction to computer networks (2360334) – Mandatory.
2. Internet Networking (2360341) – Nice to have.

Instructor

Eran Tavor (tavran@cs.technion.ac.il)

References

[1] Mininet: <https://mininet.org/>

[2] Ma, J., Dong, D., Li, C., Wu, K., Xiao, L. (2022). Evaluation of Topology-Aware All-Reduce Algorithm for Dragonfly Networks. In: Cérin, C., Qian, D., Gaudiot, J.L., Tan, G., Zuckerman, S. (eds) Network and Parallel Computing. NPC 2021. Lecture Notes in Computer Science(), vol 13152. Springer, Cham. https://doi.org/10.1007/978-3-030-93571-9_19

[3] Salimi Beni, M., Hunold, S. & Cosenza, B. Analysis and prediction of performance variability in large-scale computing systems. J Supercomput 80, 14978–15005 (2024). <https://doi.org/10.1007/s11227-024-06040-w>

[4] Zheng, Y.; Wu, J. Towards Efficient HPC: Exploring Overlap Strategies Using MPI Non-Blocking Communication. *Mathematics* **2025**, *13*, 1848. <https://doi.org/10.3390/math13111848>

[5]